

## 04 — HANA Vector Store & Embeddings

**What it is:** turns documents into searchable vectors stored natively in HANA Cloud, using HANA's REAL\_VECTOR column type and built-in COSINE\_SIMILARITY() function — the foundation for RAG (sheet 05).

### Flow

raw text → chunk (RecursiveCharacterTextSplitter) → embed (LLM embedding model)  
→ store in HANA REAL\_VECTOR column → query = embed + COSINE\_SIMILARITY search

### 1. Create the Table

```
sql = f"""
CREATE TABLE IF NOT EXISTS "{table_name}" (
    "ID"          NVARCHAR(100) NOT NULL PRIMARY KEY,
    "SOURCE_NAME" NVARCHAR(500),
    "CHUNK_INDEX" INTEGER,
    "CONTENT"     NCLOB,                -- the raw chunk text
    "METADATA"    NCLOB,                -- JSON string
    "EMBEDDING"   REAL_VECTOR({VECTOR_DIMENSION}) -- 1536 for ada-002, 3072 for 3-large
)
"""
```

### 2. Chunk

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,          # max characters per chunk – smaller = more precise search
    chunk_overlap=50,        # shared chars between chunks – avoids losing context at boundaries
    separators=["\n\n", "\n", ". ", " ", ""], # tries paragraph -> line -> sentence -> word
)
chunks = splitter.split_documents(docs)
```

### 3. Embed + Store

```
insert_sql = f"""
INSERT INTO "{table_name}" ("ID","SOURCE_NAME","CHUNK_INDEX","CONTENT","METADATA","EMBEDDING")
VALUES (?, ?, ?, ?, ?, TO_REAL_VECTOR(?)) -- TO_REAL_VECTOR converts string -> vector
"""

vectors = embedding_model.embed_documents([c.page_content for c in batch]) # batched API call
rows = [(uuid.uuid4(), c.metadata["source_name"], c.metadata["chunk_index"],
        c.page_content, json.dumps(c.metadata), str(v)) for c, v in zip(batch, vectors)]
cursor.executemany(insert_sql, rows)
conn.commit()
```

### 4. Search

```
sql = f"""
SELECT TOP {top_k} "SOURCE_NAME", "CONTENT",
    COSINE_SIMILARITY("EMBEDDING", TO_REAL_VECTOR(?)) AS SCORE -- 1.0 = identical, 0 = unrelated
FROM "{table_name}"
ORDER BY SCORE DESC
"""

query_vector = str(embedding_model.embed_query("What is SAP HANA Cloud?"))
cursor.execute(sql, (query_vector,))
```

### Key Points

- Embedding **dimension must match** between the table's REAL\_VECTOR(N) and the model used (ada-002=1536, text-embedding-3-large=3072) — mismatches error at insert time.
- Vectors are inserted as **strings** via TO\_REAL\_VECTOR(?) — the driver doesn't send a native vector type, HANA parses the string server-side.

- `chunk_overlap` prevents a sentence that spans a chunk boundary from losing meaning in both halves.
- `COSINE_SIMILARITY` measures the angle between vectors — insensitive to text length, which is why it's preferred over Euclidean distance for text embeddings.
- Batch embedding calls (`batch_size=50`) to avoid provider rate limits on large document sets.

## Interview Q&A

**Q: Why cosine similarity instead of Euclidean distance for text embeddings?** A: Cosine similarity measures direction (semantic meaning), not magnitude — two chunks of very different length but similar meaning still score close to 1.0.

**Q: What happens if chunk\_size is too large or too small?** A: Too large → embeddings blur multiple topics together, hurting search precision. Too small → each chunk loses surrounding context, and you need more of them (higher retrieval + LLM cost) to get a complete answer.

**Q: How is this different from a dedicated vector database (Pinecone, Weaviate)?** A: HANA Vector Engine keeps vectors alongside your relational/transactional data in the same database — no separate system to sync, and you can combine vector search with normal SQL filters (`WHERE`, `JOIN`) in one query.